详细设计说明书

文档信息

项目	内容
项目名称	宝可梦对战系统 v2.0
文档版本	V1.0
编写日期	2026 年 4 月 8 日
适用阶段	Sprint 2 重构交付

一、系统架构概述

1.1 重构前后架构对比

• 重构前架构（过程式）：

```
main.c (835 行)
├── 数据结构定义 (Pokemon, Skill, Battle)
├── 全局变量 (type_chart)
├── 初始化函数 (init_pokemon, add_skill, create_xxx)
├── 显示函数 (show_hp_bar, show_pokemon_info, show_battle)
├── 战斗逻辑 (run_battle, attack, check_win)
├── 伤害计算 (calc_dmg, check_hit, check_crit)
└── 状态管理 (apply_status, handle_status_damage, is_asleep)
```

• 重构后架构（面向对象）：

```
src/
├── domain/
│   ├── Pokemon.java        # 宝可梦实体
│   ├── Skill.java          # 技能实体
│   └── enums/              # 枚举类型
├── battle/
│   ├── Battle.java         # 战斗上下文（核心控制类）
│   └── BattleResult.java   # 战斗结果
├── strategy/
│   ├── damage/             # 伤害计算策略
│   ├── type/               # 属性克制策略
│   └── ai/                 # AI 决策策略
├── factory/                # 宝可梦创建工厂
└── observer/               # 战斗事件观察者
```

二、核心控制类详细设计

2.1 Battle 类（战斗上下文）

2.1.1 类定义与方法签名

```
package battle;

import domain.Pokemon;
import domain.Skill;
```

```java
import strategy.damage.IDamageCalculator;
import strategy.ai.IAIStrategy;
import observer.IBattleObserver;
import java.util.List;
import java.util.ArrayList;

/**
 * 战斗上下文类 - 核心控制类
 * 职责：管理战斗流程、回合执行、状态同步
 */
public class Battle {

    // ========== 私有字段 ==========
    private Pokemon player1;                    // 玩家 1 宝可梦
    private Pokemon player2;                    // 玩家 2/AI 宝可梦
    private int turn;                           // 当前回合数
    private boolean isFinished;                 // 战斗是否结束
    private BattleResult result;                // 战斗结果

    private IDamageCalculator damageCalculator;   // 伤害计算策略
    private IAIStrategy aiStrategy;               // AI 决策策略
    private List<IBattleObserver> observers;      // 观察者列表

    // ========== 构造方法 ==========

    /**
     * 构造战斗实例
     * @param p1 玩家 1 的宝可梦
     * @param p2 玩家 2 的宝可梦
     * @param damageCalculator 伤害计算器
     * @param aiStrategy AI 策略
     */
    public Battle(Pokemon p1, Pokemon p2,
                  IDamageCalculator damageCalculator,
                  IAIStrategy aiStrategy) {
        this.player1 = p1;
        this.player2 = p2;
        this.damageCalculator = damageCalculator;
        this.aiStrategy = aiStrategy;
        this.turn = 1;
        this.isFinished = false;
        this.observers = new ArrayList<>();
    }

    // ========== 公共方法 ==========

    /**
     * 启动战斗主循环
     */
```

```
public void start() {
    notifyStart();
    while (!isFinished) {
        executeTurn();
    }
    notifyEnd();
}

/**
 * 注册战斗事件观察者
 * @param observer  观察者实例
 */
public void attachObserver(IBattleObserver observer) {
    if (observer != null && !observers.contains(observer)) {
        observers.add(observer);
    }
}

/**
 * 移除观察者
 * @param observer  观察者实例
 */
public void detachObserver(IBattleObserver observer) {
    observers.remove(observer);
}

/**
 * 获取当前回合数
 * @return  回合数
 */
public int getTurn() {
    return turn;
}

/**
 * 获取战斗结果
 * @return BattleResult  对象
 */
public BattleResult getResult() {
    return result;
}

// ==========  私有方法  ==========

/**
 * 执行单个回合
 */
private void executeTurn() {
    // 1. 获取双方技能选择
```

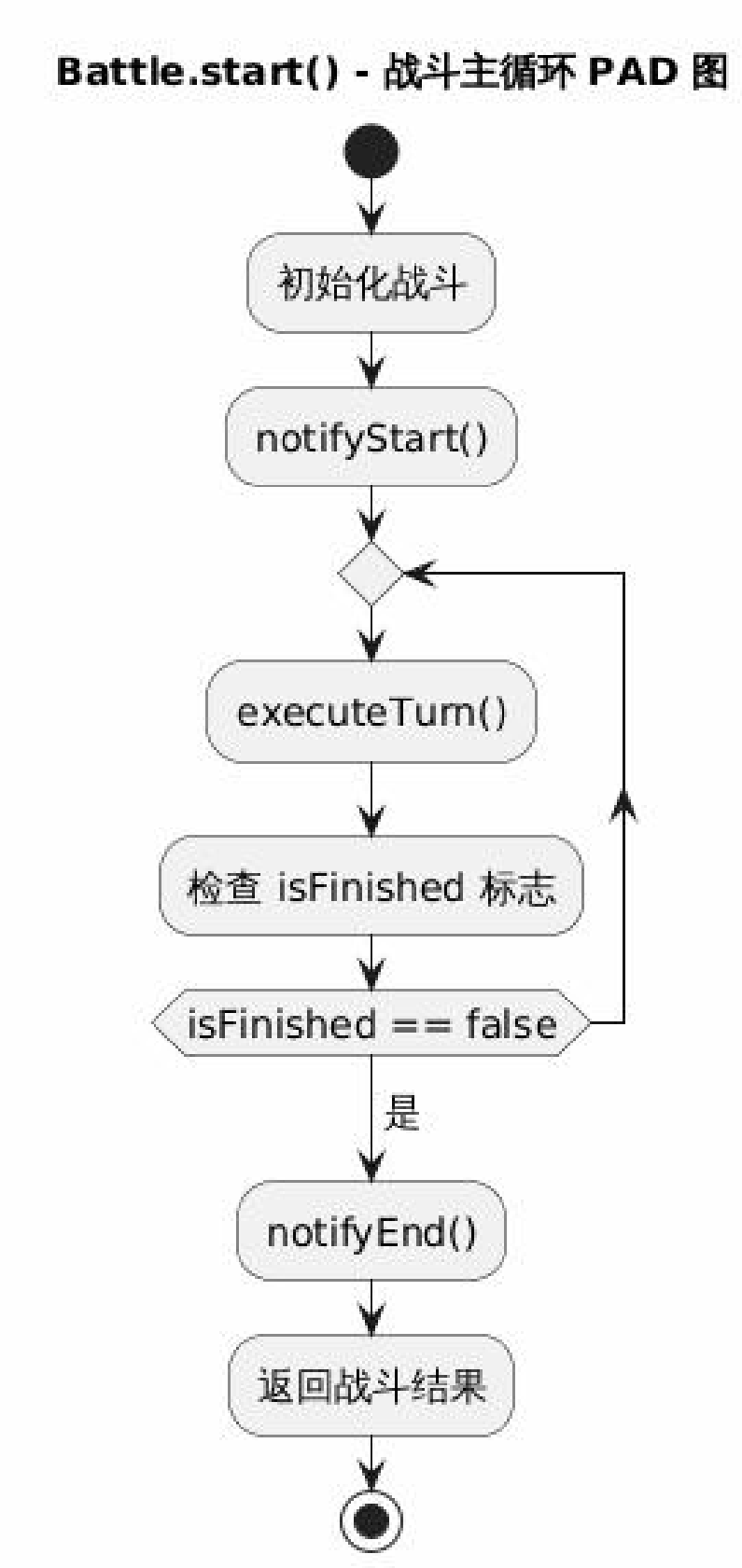
```
        Skill p1Skill = getPlayer1Choice();
        Skill p2Skill = aiStrategy.selectSkill(player2, player1);

        // 2. 判定先手顺序
        int first = determineTurnOrder();

        // 3. 执行攻击（按速度顺序）
        if (first == 1) {
            executeSingleAttack(player1, player2, p1Skill);
            if (!player2.isFainted()) {
                executeSingleAttack(player2, player1, p2Skill);
            }
        } else {
            executeSingleAttack(player2, player1, p2Skill);
            if (!player1.isFainted()) {
                executeSingleAttack(player1, player2, p1Skill);
            }
        }

        // 4. 处理状态异常
        handleStatusEffects();

        // 5. 检查胜负
        checkAndSetResult();

        // 6. 回合递增
        turn++;
    }

    /**
     * 执行单次攻击
     * @param attacker 攻击方
     * @param defender 防御方
     * @param skill 使用的技能
     */
    private void executeSingleAttack(Pokemon attacker, Pokemon defender, Skill skill) {
        // 检查睡眠状态
        if (attacker.isAsleep()) {
            notifyStatusSkip(attacker);
            attacker.wakeUp();
            return;
        }

        // 消耗 PP
        skill.use();

        // 通知攻击事件
        notifyAttack(attacker, skill);
```

```
        // 计算伤害
        DamageResult dmgResult = damageCalculator.calculate(attacker, defender, skill);

        // 应用伤害
        defender.takeDamage(dmgResult.getDamage());

        // 通知伤害事件
        notifyDamage(defender, dmgResult);

        // 附加状态异常
        if (dmgResult.hasStatusEffect()) {
            defender.addStatus(dmgResult.getStatusEffect(), dmgResult.getStatusTurns());
            notifyStatusChange(defender, dmgResult.getStatusEffect());
        }
    }

    /**
     * 判定先手顺序
     * @return 1 表示玩家 1 先手，2 表示玩家 2 先手
     */
    private int determineTurnOrder() {
        int speed1 = player1.getEffectiveSpeed();
        int speed2 = player2.getEffectiveSpeed();

        if (speed1 >= speed2) {
            return 1;
        } else {
            return 2;
        }
    }

    /**
     * 处理状态异常效果
     */
    private void handleStatusEffects() {
        handleStatusForPokemon(player1);
        handleStatusForPokemon(player2);
    }

    /**
     * 处理单个宝可梦的状态
     * @param pokemon 宝可梦实例
     */
    private void handleStatusForPokemon(Pokemon pokemon) {
        if (pokemon.hasStatus()) {
            int damage = pokemon.getStatus().getDamagePerTurn(pokemon);
            if (damage > 0) {
                pokemon.takeDamage(damage);
                notifyStatusDamage(pokemon, damage);
```

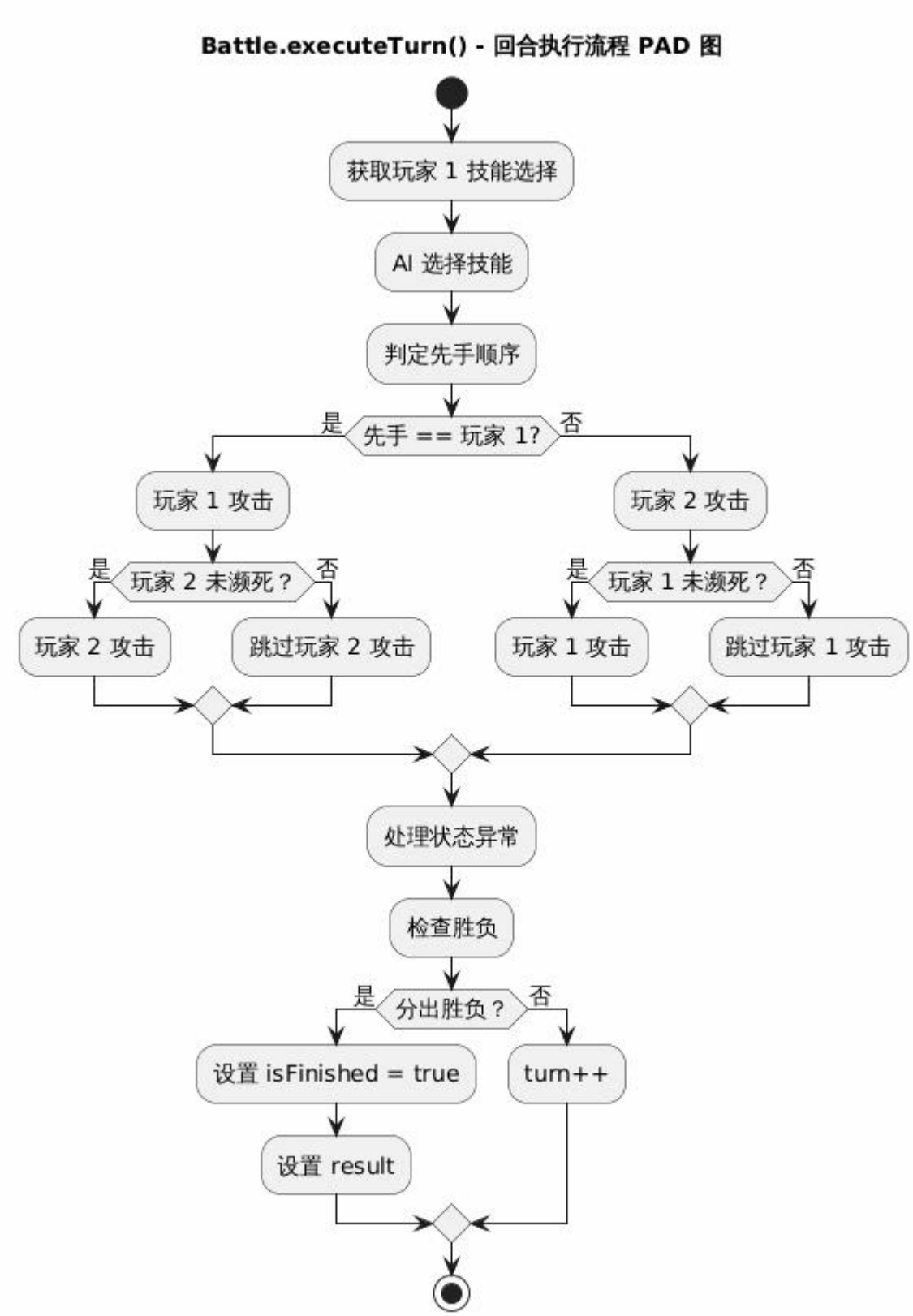
```
            }
            pokemon.decrementStatusTurns();
            if (pokemon.getStatusExpired()) {
                notifyStatusCured(pokemon);
            }
        }
    }

    /**
     * 检查并设置战斗结果
     */
    private void checkAndSetResult() {
        boolean p1Fainted = player1.isFainted();
        boolean p2Fainted = player2.isFainted();

        if (p1Fainted && p2Fainted) {
            result = new BattleResult(BattleResult.Type.DRAW, null, turn);
            isFinished = true;
        } else if (p1Fainted) {
            result = new BattleResult(BattleResult.Type.PLAYER2_WIN, player2, turn);
            isFinished = true;
        } else if (p2Fainted) {
            result = new BattleResult(BattleResult.Type.PLAYER1_WIN, player1, turn);
            isFinished = true;
        }
    }

    // ========== 观察者通知方法 ==========

    private void notifyStart() {
        observers.forEach(o -> o.onBattleStart(player1, player2));
    }

    private void notifyAttack(Pokemon attacker, Skill skill) {
        observers.forEach(o -> o.onAttack(attacker, skill));
    }

    private void notifyDamage(Pokemon defender, DamageResult result) {
        observers.forEach(o -> o.onDamage(defender, result));
    }

    private void notifyStatusChange(Pokemon pokemon, StatusCondition status) {
        observers.forEach(o -> o.onStatusChange(pokemon, status));
    }

    private void notifyEnd() {
        observers.forEach(o -> o.onBattleEnd(result));
    }
}
```

2.2 核心算法 PAD 图

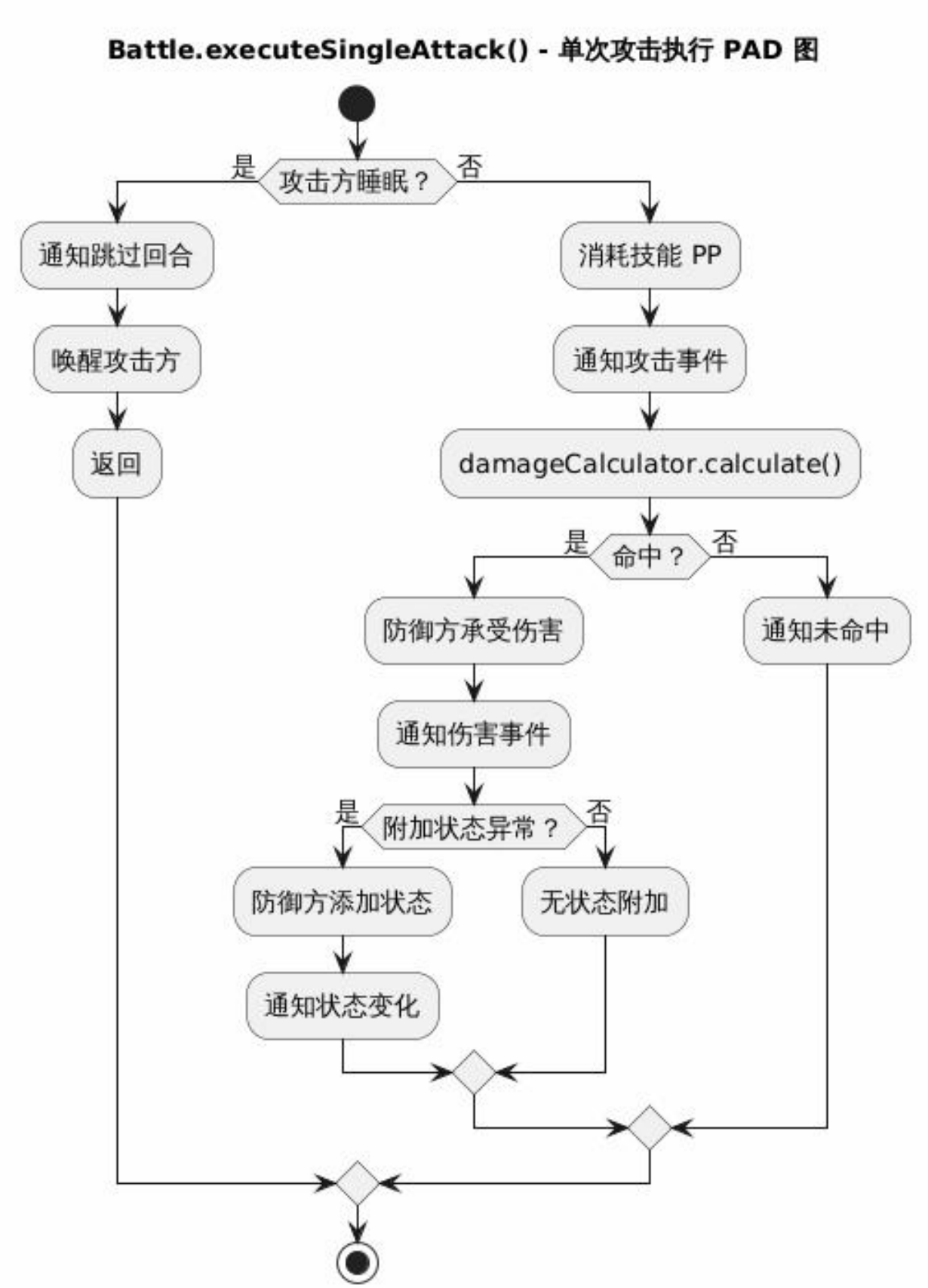
• **算法 1：战斗主循环 (Battle.start)**



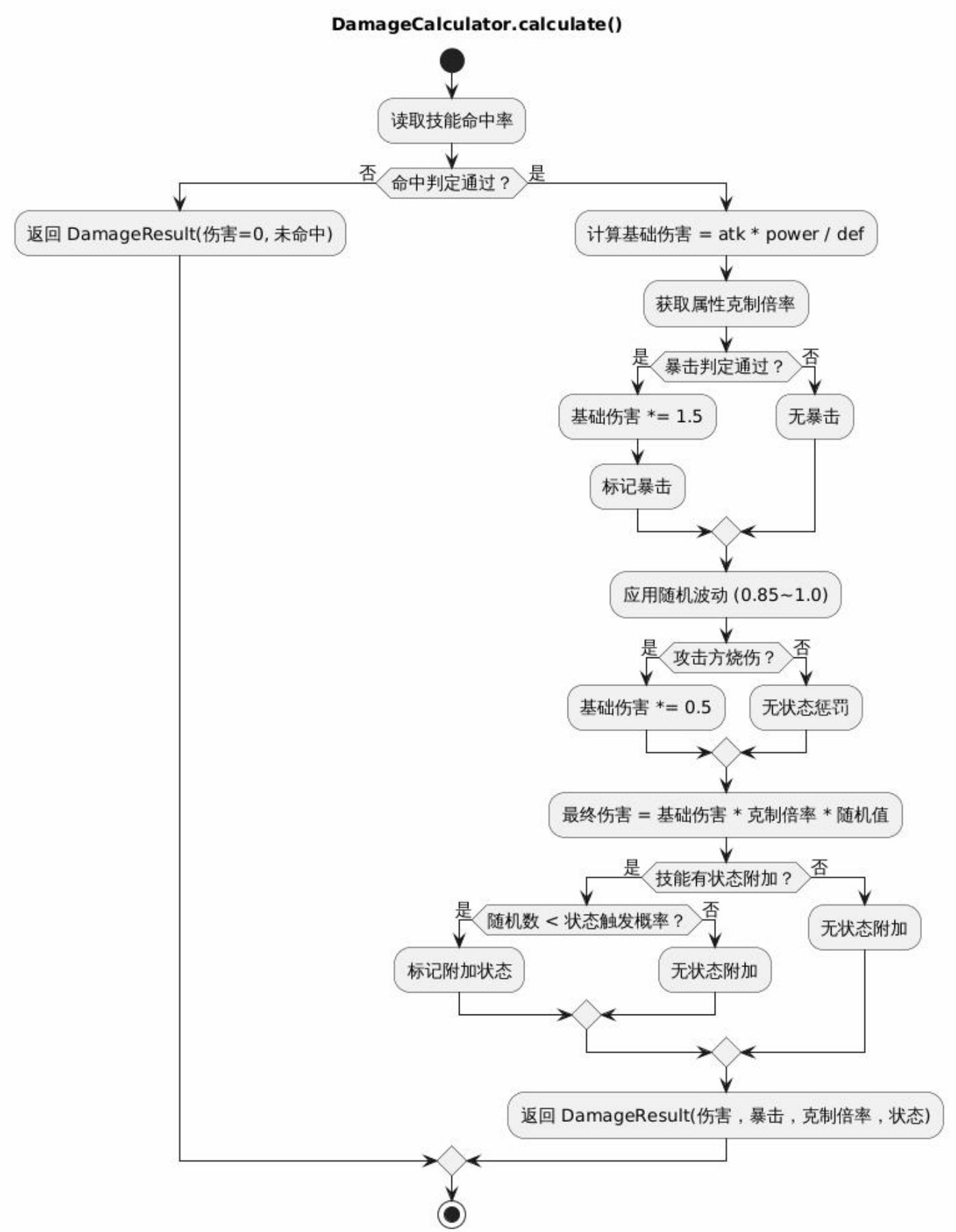
• 算法 2：回合执行流程 (Battle.executeTurn)



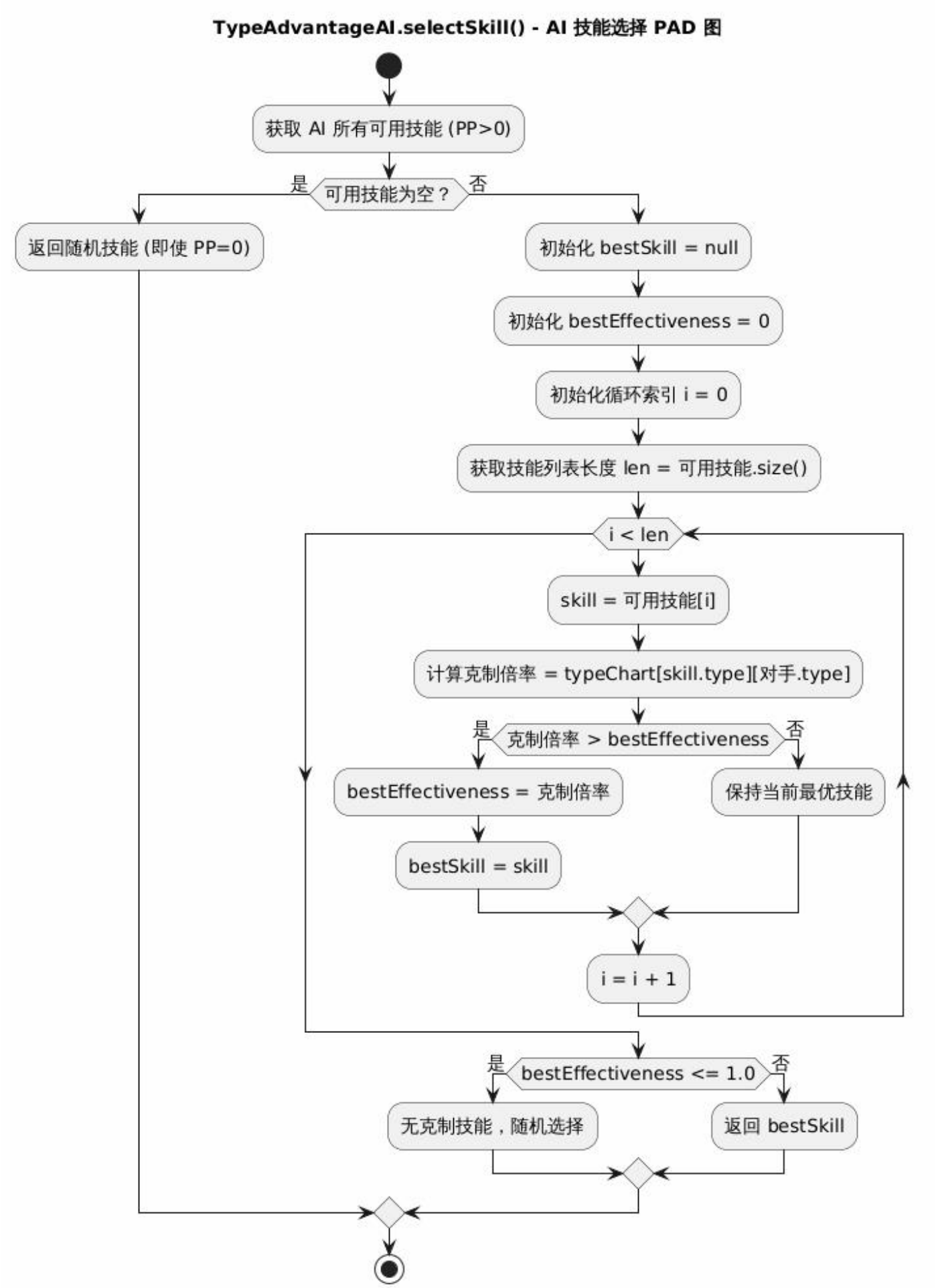
• 算法 3：单次攻击执行 (Battle.executeSingleAttack)



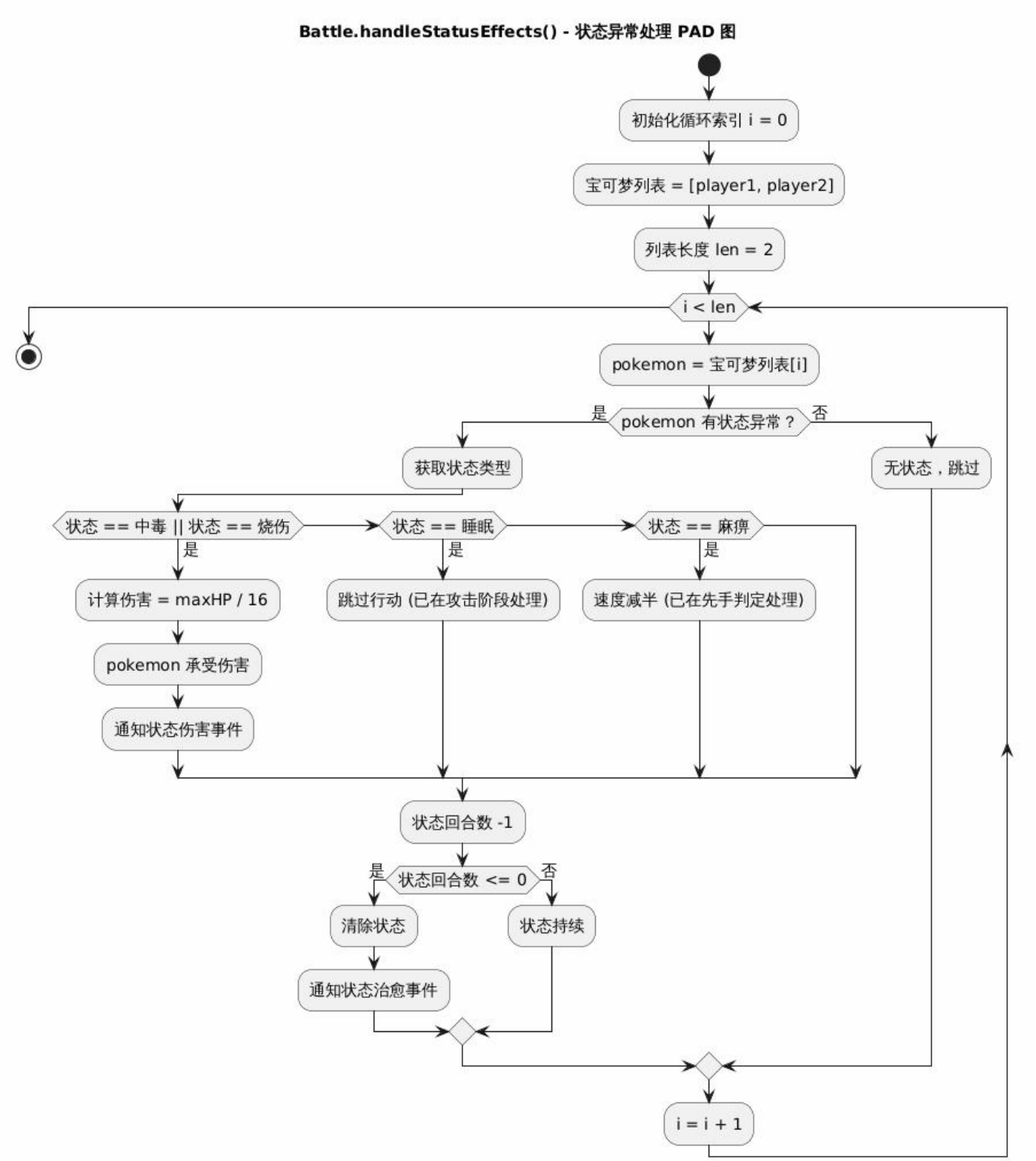
• 算法 4：伤害计算 (DamageCalculator.calculate)



• 算法 5：AI 技能选择 (TypeAdvantageAI.selectSkill)

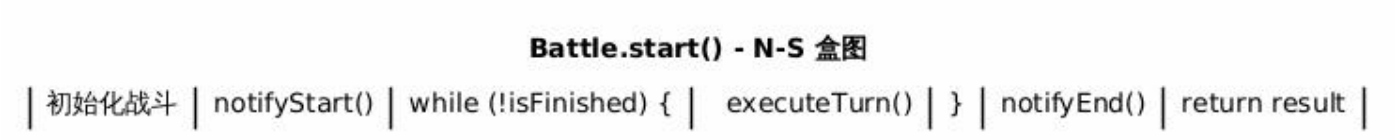


• 算法 6：状态异常处理 (Battle.handleStatusEffects)



2.3 N-S 盒图（Nassi-Shneiderman）

• N-S 盒图 1：战斗主循环



• N-S 盒图 2：回合执行（PlantText）

```
@startuml
skinparam defaultFontSize 14
skinparam defaultFontName "Microsoft YaHei"
skinparam backgroundColor #FEFEFE

title Battle.executeTurn() - N-S 盒图

| p1Skill = getPlayer1Choice() | p2Skill = aiStrategy.select() |
| first = determineTurnOrder() |
| if (first == 1) |
|     executeAttack(p1, p2, p1Skill) |
|     if (!p2.fainted) |
|         executeAttack(p2, p1, p2Skill) |
|     else |
|         跳过 |
|     endif |
| else |
|     executeAttack(p2, p1, p2Skill) |
|     if (!p1.fainted) |
|         executeAttack(p1, p2, p1Skill) |
|     else |
|         跳过 |
|     endif |
| endif |
| handleStatusEffects() |
| checkAndSetResult() |
| if (finished) | turn++ |
|     isFinished=true | |
|     result=xxx | |
| endif |

@enduml
```

• N-S 盒图 3：伤害计算（PlantText）

```
@startuml
skinparam defaultFontSize 14
skinparam defaultFontName "Microsoft YaHei"
skinparam backgroundColor #FEFEFE

title DamageCalculator.calculate() - N-S 盒图
```

```
| if (!checkHit(accuracy)) |
|     return DamageResult(0, MISS) |
| else |
|     base = atk * power / def |
|     effectiveness = typeChart[atkType][defType] |
|     if (checkCrit(critRate)) |
|         base *= 1.5 |
|         isCrit = true |
|     else |
|         isCrit = false |
|     endif |
|     rnd = 0.85 + rand() * 0.15 |
|     if (attacker.burned) |
|         base *= 0.5 |
|     endif |
|     damage = base * effectiveness * rnd |
|     if (skill.hasStatus && rand() < chance) |
|         status = skill.status |
|     else |
|         status = NONE |
|     endif |
|     return DamageResult(damage, isCrit, effectiveness, status) |
| endif |

@enduml
```

三、跨类调用方法签名规范

3.1 领域层 → 策略层调用

调用方	被调用方	方法签名	返回值	说明
Battle	IDamageCalculator	calculate(Pokemon atk, Pokemon def, Skill skill)	DamageResult	计算伤害并返回完整结果
Battle	IAIStrategy	selectSkill(Pokemon ai, Pokemon opponent)	Skill	AI 选择最优技能
DamageCalculator	ITypeStrategy	getEffectiveness(PokemonType atk, PokemonType def)	float	获取属性克制倍率

3.2 战斗层 → 领域层调用

调用方	被调用方	方法签名	返回值	说明
Battle	Pokemon	takeDamage(int damage)	int	承受伤害，返回实际伤害值
Battle	Pokemon	isFainted()	boolean	检查是否濒死
Battle	Pokemon	isAsleep()	boolean	检查是否睡眠
Battle	Pokemon	addStatus(StatusCondition status, int turns)	void	添加状态异常
Battle	Pokemon	getEffectiveSpeed()	int	获取有效速度（含麻痹减速）
Battle	Skill	use()	boolean	使用技能，消耗 PP
Battle	Skill	isAvailable()	boolean	检查 PP 是否可用

3.3 战斗层 → 观察者层调用

调用方	被调用方	方法签名	返回值	说明
Battle	IBattleObserver	onBattleStart(Pokemon p1, Pokemon p2)	void	战斗开始通知
Battle	IBattleObserver	onAttack(Pokemon atk, Skill skill)	void	攻击动作通知
Battle	IBattleObserver	onDamage(Pokemon def, DamageResult result)	void	伤害结算通知
Battle	IBattleObserver	onStatusChange(Pokemon pkmn, StatusCondition status)	void	状态变化通知
Battle	IBattleObserver	onBattleEnd(BattleResult result)	void	战斗结束通知

3.4 工厂层 → 领域层调用

调用方	被调用方	方法签名	返回值	说明
IPokemonFactory	Pokemon	createPokemon(int level)	Pokemon	创建指定等级的宝可梦
PokemonFactoryRegistry	IPokemonFactory	register(String name, IPokemonFactory factory)	void	注册工厂
PokemonFactoryRegistry	IPokemonFactory	create(String name, int level)	Pokemon	通过名称创建宝可梦

四、数据结构定义

4.1 DamageResult（伤害计算结果）

```
package battle;

import domain.StatusCondition;

/**
 * 伤害计算结果 DTO
 */
public class DamageResult {

    public enum HitType {
        HIT,            // 命中
        MISS,           // 未命中
        IMMUNE          // 免疫
    }

    private int damage;                     // 最终伤害值
    private boolean isCritical;             // 是否暴击
    private float effectiveness;            // 属性克制倍率
    private HitType hitType;                // 命中类型
    private StatusCondition statusEffect;   // 附加状态
    private int statusTurns;                // 状态持续回合数

    public DamageResult(int damage, boolean isCrit, float eff, HitType type) {
        this.damage = damage;
        this.isCritical = isCrit;
        this.effectiveness = eff;
        this.hitType = type;
        this.statusEffect = StatusCondition.NONE;
        this.statusTurns = 0;
    }
```

```
    public boolean hasStatusEffect() {
        return statusEffect != StatusCondition.NONE;
    }

    // Getters & Setters...
}
```

4.2 BattleResult（战斗结果）

```
package battle;

import domain.Pokemon;

/**
 * 战斗结果 DTO
 */
public class BattleResult {

    public enum Type {
        PLAYER1_WIN,
        PLAYER2_WIN,
        DRAW
    }

    private Type type;                  // 胜负类型
    private Pokemon winner;             // 获胜者（平局为 null）
    private int turnCount;              // 总回合数

    public BattleResult(Type type, Pokemon winner, int turnCount) {
        this.type = type;
        this.winner = winner;
        this.turnCount = turnCount;
    }

    public boolean isDraw() {
        return type == Type.DRAW;
    }

    // Getters...
}
```

五、接口定义规范

5.1 IDamageCalculator（伤害计算器接口）

```
package strategy.damage;

import domain.Pokemon;
import domain.Skill;
import battle.DamageResult;

/**
 * 伤害计算器接口
```

```
 * 实现策略模式，支持不同伤害公式
 */
public interface IDamageCalculator {

    /**
     * 计算伤害
     * @param attacker 攻击方
     * @param defender 防御方
     * @param skill 使用的技能
     * @return 伤害计算结果
     */
    DamageResult calculate(Pokemon attacker, Pokemon defender, Skill skill);

    /**
     * 获取属性克制倍率
     * @param attackType 攻击技能类型
     * @param defendType 防御方属性
     * @return 克制倍率 (0.0~4.0)
     */
    float getEffectiveness(PokemonType attackType, PokemonType defendType);
}
```

5.2 IAIStrategy（AI 策略接口）

```
package strategy.ai;

import domain.Pokemon;
import domain.Skill;

/**
 * AI 决策策略接口
 * 实现不同难度的 AI 行为
 */
public interface IAIStrategy {

    /**
     * 选择技能
     * @param aiPokemon AI 的宝可梦
     * @param opponent 对手的宝可梦
     * @return 选择的技能
     */
    Skill selectSkill(Pokemon aiPokemon, Pokemon opponent);
}
```

5.3 IBattleObserver（战斗观察者接口）

```
package observer;

import domain.Pokemon;
import domain.Skill;
import domain.StatusCondition;
import battle.BattleResult;
import battle.DamageResult;
```

```java
/**
 * 战斗事件观察者接口
 * 实现观察者模式，解耦战斗逻辑与显示/日志
 */
public interface IBattleObserver {

    /**
     * 战斗开始
     */
    void onBattleStart(Pokemon player1, Pokemon player2);

    /**
     * 攻击动作
     */
    void onAttack(Pokemon attacker, Skill skill);

    /**
     * 伤害结算
     */
    void onDamage(Pokemon defender, DamageResult result);

    /**
     * 状态变化
     */
    void onStatusChange(Pokemon pokemon, StatusCondition status);

    /**
     * 战斗结束
     */
    void onBattleEnd(BattleResult result);
}
```

六、异常处理规范

6.1 自定义异常类

```java
package exception;

/**
 * 宝可梦不存在异常
 */
public class PokemonNotFoundException extends RuntimeException {
    public PokemonNotFoundException(String name) {
        super("未找到宝可梦：" + name);
    }
}

/**
 * 技能不可用异常
 */
```

```
public class SkillNotAvailableException extends RuntimeException {
    public SkillNotAvailableException(String skillName) {
        super("技能不可用：" + skillName);
    }
}

/**
 * 战斗已结束异常
 */
public class BattleFinishedException extends RuntimeException {
    public BattleFinishedException() {
        super("战斗已经结束，无法继续操作");
    }
}
```

七、McCabe 复杂度计算

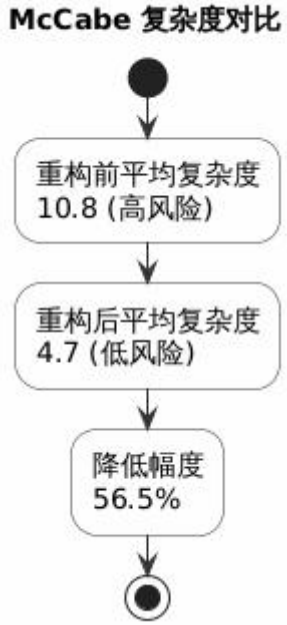
7.1 重构前复杂度统计

函数名	行数	判定节点数	McCabe 复杂度	等级
run_battle()	120	15	16	高风险
attack()	65	10	11	高风险
calc_dmg()	35	7	8	中等风险
calc_type_effectiveness()	40	18	19	高风险
handle_status_damage()	15	4	5	中等风险
select_skill()	25	5	6	中等风险
平均复杂度	-	-	10.8	高风险

7.2 重构后复杂度统计

类名	方法名	行数	判定节点数	McCabe 复杂度	等级
Battle	executeTurn()	25	5	6	低风险
Battle	executeSingleAttack()	20	4	5	低风险
PhysicalDamageCalculator	calculate()	18	5	6	低风险
TypeAdvantageAI	selectSkill()	15	3	4	低风险
Battle	handleStatusEffects()	12	3	4	低风险
PokemonFactoryRegistry	create()	8	2	3	低风险
平均复杂度	-	-	4.7		低风险

7.3 复杂度对比图



八、版本历史

版本	日期	修改内容	修改人
V1.0	2026-04-08	初始版本，完成核心控制类设计	杨家鹏